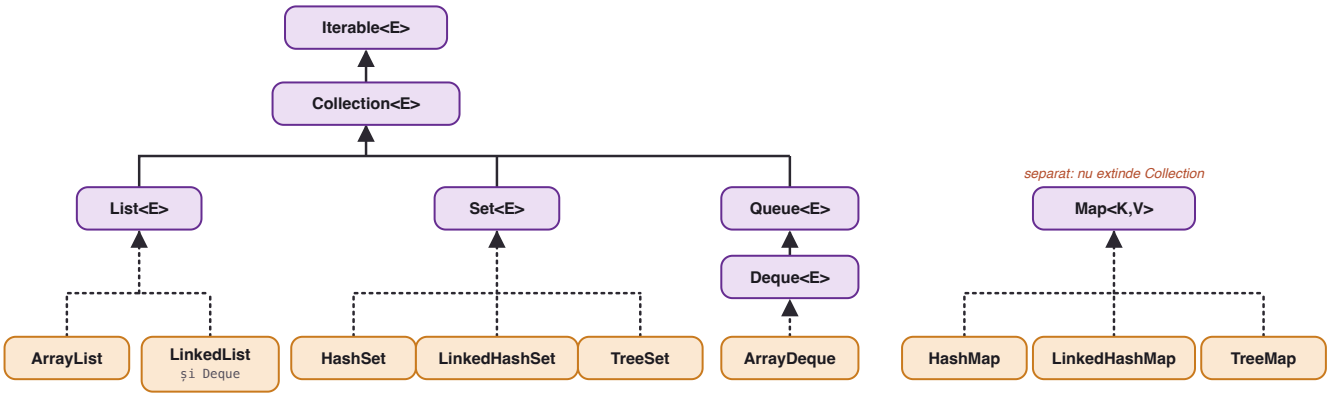
# Colecțiile din JDK — Schema

Cine pe cine extinde, în `java.util` · scrii mereu `Interfata<E> x = new Implementare<>();` · Adrian

Interfață · Clasă · extinde (interfață din interfață) · implementează

## 1. Cine pe cine extinde

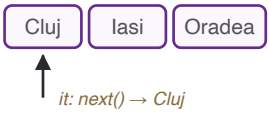



## 2. Ce colecție alegi

| Întrebarea ta | Colecția | Ce folosești |
|---|---|---|
| „Am o înșiruire în ordine, cu poziții” | `ArrayList<E>` | `add, get(i), set, remove(i), size` |
| „Adaug și scot des la capete” | `ArrayDeque<E>` | `offerFirst, offerLast, pollFirst, pollLast` |
| „Inserez/scot în mijlocul unei liste mari” | `LinkedList<E>` | `add, remove, addFirst, removeLast` |
| „Am văzut deja asta?” / „fără dubluri” | `HashSet<E>` | `add` (false dacă exista), `contains` |
| „Fără dubluri, în ordinea în care au venit” | `LinkedHashSet<E>` | `ca la HashSet; for-each în ordinea inserării` |
| „Fără dubluri și sortat” | `TreeSet<E>` | `add, contains, first, last; for-each crescător` (elemente cu `Comparable` / `Comparator`) |
| „Caut ceva după o cheie (nume, cod)” | `HashMap<K,V>` | `put, get, getOrDefault, containsKey` |
| „Caut după cheie, afișez în ordinea adăugării” | `LinkedHashMap<K,V>` | `ca la HashMap; parcurgerea în ordinea inserării` |
| „Caut după cheie, afișez cheile sortate” | `TreeMap<K,V>` | `ca la HashMap; firstKey, lastKey` |
| „Procesez în ordinea sosirii” | `Queue<E> = ArrayDeque` | `offer, poll, peek` |
| „Mă întorc la ultimul lucru” | `Deque<E> = ArrayDeque` | `push, pop, peek` |
| „Știu sigur câte sunt, nu se schimbă” | `tablou E[]` | `t[i], t.length, Arrays.sort` |

## Iterable<E>

Tot ce merge în `for-each`. `Collection` îl extinde; `Map` nu.




hasNext() → true
după Oradea:
hasNext() → false

| | |
|---|---|
| `for (E x : c)` | = iterator() + hasNext() + next() |
| `iterator()` | → Iterator<E> |
| `it.hasNext()` | mai e ceva? → boolean |
| `it.next()` | următorul element |
| `it.remove()` | scoate ce a dat next() |

⚠ `next()` când `hasNext()` e `false` → `NoSuchElementException`. În timpul parcurgerii scoți doar cu `it.remove()`.

## Collection<E>

Interfața comună: orice colecție, în afară de Map.




add("Zoe") → true
size() → 4
*fără get(i)*

| | |
|---|---|
| `add(x)` | → boolean: s-a adăugat? |
| `size()` | câte sunt |
| `contains(x)` | există? → boolean |
| `remove(x)` | prima apariție → boolean |
| `isEmpty() / clear()` | verifică / golește |

⚠ `add` întoarce `false` dacă colecția nu s-a schimbat (duplicat într-un `Set`).

`Collection<String> c = new ArrayList<>();`
stânga = interfața (ce poți face), dreapta = implementarea.

## List<E>

Înșiruire cu poziții de la 0; duplicatele sunt permise.




add("Zoe")
size() = 3

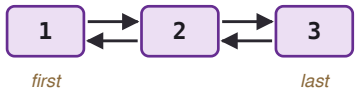
| | |
|---|---|
| `add(x)` | la sfârșit → true |
| `add(i, x)` | pe poziția i |
| `get(i)` | citește de pe i |
| `set(i, x)` | înlocuiește → vechea valoare |
| `indexOf(x)` | poziția sau -1 |
| `remove(int i)` | scoate de pe poziția i → elementul |
| `Collections.sort(l)` | pe loc, elemente comparabile |

⚠ `get(i)` în afara listei → **IndexOutOfBoundsException** (nu `null`).

`List<Integer> l` = [10, 20, 30]
`l.remove(1)` scoate poziția 1 → [10, 30]
`l.remove(Integer.valueOf(10))` scoate valoarea 10

## LinkedList<E>

Noduri legate; rapidă la capete. Este și `List`, și `Deque`.

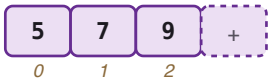



| | |
|---|---|
| `addFirst(x)` | la început |
| `addLast(x)` | la sfârșit |
| `removeFirst()` | scoate primul → elementul |
| `removeLast()` | scoate ultimul → elementul |
| `getFirst()` | se uită la primul |

⚠ `removeFirst()` / `getFirst()` pe listă goală → **NoSuchElementException**. `get(i)` parcurge nodurile (lent).

## ArrayList<E>

Tablou care crește singur; get(i) rapid.




add(x) pune aici
size() = 3

| | |
|---|---|
| `new ArrayList<>()` | listă goală |
| `new ArrayList<>(alta)` | copie modificabilă |
| `add / get / set` | ca la List |
| `addAll(alta)` | adaugă toate → boolean |

⚠ `remove` / `add` într-un `for-each` pe ea → **ConcurrentModificationException**.

implementează `List` · `Collection`

## Arrays.asList(...)

Listă cu dimensiune fixă, peste valorile date.




add → eroare
set merge

| | |
|---|---|
| `set(i, x)` | merge: schimbă o valoare |
| `get(i) / size()` | merg |
| `add(x)` | UnsupportedOperationException |
| `remove(x)` | existent → UnsupportedOperationException; inexistent → false |

⚠ Nu-i poți schimba lungimea. Listă modificabilă: `new ArrayList<>(Arrays.asList("Ana", "Ion"))`.

## Set<E>

Mulțime: fiecare valoare o singură dată.



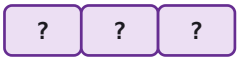

*A=[1,2,3] B=[2,3,4]*
addAll(B) → 1,2,3,4
retainAll(B) → 2,3
removeAll(B) → 1

| | |
|---|---|
| `add(x)` | false dacă exista deja |
| `contains(x)` | există? → boolean |
| `addAll(o)` | reuniune, pe loc |
| `retainAll(o)` | intersecție, pe loc |
| `removeAll(o)` | diferență, pe loc |

⚠ Cele trei **modifică** mulțimea ta. Copie întâi: `new HashSet<>(a)`. Fără `get(i)`.

## HashSet<E>

Rapidă la `contains`; ordinea o decide hash-ul.




*adaugi Maria, Ana, Zoe → ordine imprevizibilă*

| | |
|---|---|
| `add(x)` | false dacă exista |
| `contains(x)` | direct, rapid |
| `remove(x)` | → boolean |
| `size()` | câte sunt |

⚠ Nu te baza pe ordinea de la `for-each`: nu e inserarea, nu e sortarea.

## LinkedHashSet<E>

Ca `HashSet`, dar ține ordinea inserării.




*adaugi Maria, Ana, Zoe → Maria, Ana, Zoe*

| | |
|---|---|
| `add(x)` | false dacă exista |
| `contains(x)` | există? |
| `remove(x)` | → boolean |
| `for-each` | în ordinea inserării |

⚠ Re-adăugarea unui element existent nu-l mută la sfârșit.

## TreeSet<E>

Fără dubluri, mereu sortată crescător.




*adaugi Maria, Ana, Zoe → Ana, Maria, Zoe*

| | |
|---|---|
| `add(x)` | false dacă exista |
| `first() / last()` | cel mai mic / cel mai mare |
| `contains(x)` | există? |
| `remove(x)` | → boolean |

⚠ Fără `get(i)`. Elementele trebuie să fie comparabile (`String`, `Integer`) — vezi cărțile `Comparable` / `Comparator`.

## Map<K, V>

Perechi cheie → valoare; cheile sunt unice.




d.get("Ana") → 9 d.get("Dan") → null

| | |
|---|---|
| `put(k, v)` | adaugă/suprascrie → vechea valoare |
| `get(k)` | valoarea sau null |
| `getOrDefault(k, d)` | valoarea sau d |
| `putIfAbsent(k, v)` | nu suprascrie |
| `containsKey(k)` | există cheia? |
| `remove(k)` | → valoarea scoasă |

⚠ `get` pe cheie lipsă → `null`, nu excepție.

## HashMap<K, V>

Căutare rapidă după cheie; fără ordine garantată.




put("Ana", 5) → 9 (vechea)
put("Dan", 1) → null (cheie nouă)

| | |
|---|---|
| `put / get` | ca la Map |
| `keySet()` | cheile |
| `values()` | valorile |
| `size()` | câte perechi |

⚠ O cheie apare o singură dată: `put` pe cheie existentă îi schimbă valoarea.

## LinkedHashMap<K, V>

Ca `HashMap`, dar parcurgerea e în ordinea inserării.




*adaugi Zoe, Ana, Ion*
parcurgi: Zoe, Ana, Ion

| | |
|---|---|
| `put / get` | ca la Map |
| `entrySet()` | perechi, în ordinea inserării |
| `keySet()` | chei, în ordinea inserării |

⚠ `put` pe cheie existentă nu o mută la sfârșit.

## TreeMap<K, V>

Cheile mereu sortate crescător.




*adaugi Zoe, Ana, Ion*
parcurgi: Ana, Ion, Zoe

| | |
|---|---|
| `put / get` | ca la Map |
| `firstKey()` | cea mai mică cheie |
| `lastKey()` | cea mai mare cheie |
| `entrySet()` | perechi, cu cheile crescător |

⚠ Cheia `null` nu merge (**NullPointerException**); cheile trebuie comparabile — vezi `Comparable` / `Comparator`.

## Map: parcurgere + dezambalare

Un `Map` nu are `for-each` direct: parcurgi perechile.

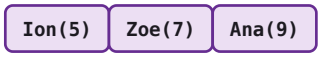
```
for (Map.Entry<String, Integer> e :
      d.entrySet()) {
  e.getKey() + " " + e.getValue()
}
```

**entrySet()** = mulțimea perechilor; `getKey()` / `getValue()` citesc o pereche.

⚠ `int n = d.get("Dan");` cu cheie lipsă → **NullPointerException**: `null` nu intră într-un `int`. Folosește `getOrDefault("Dan", 0)`.

## Comparable<T>

Ordinea naturală a clasei, scrisă în clasă.




compareTo → după notă, crescător

*a.compareTo(b) < 0 → a vine înaintea lui b*

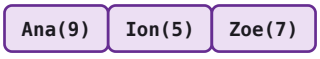
| | |
|---|---|
| `int compareTo(T altul)` | negativ / 0 / pozitiv |
| `Integer.compare(a, b)` | forma corectă, fără a - b |
| `Collections.sort(lista)` | folosește compareTo |
| `new TreeSet<>() / TreeMap` | ordine = compareTo |
| `first() / last()` | după compareTo |

```
class Elev implements
    Comparable<Elev> {
  public int compareTo(Elev o) {
    return Integer.compare(
        nota, o.nota);
  }
}
```

⚠ `a - b` poate depăși `int` ; scrii `Integer.compare` .

## Comparator<T>

O ordine din afară, într-o clasă proprie.




DupaNume → Ana, Ion, Zoe

*DupaNotaDesc → Ana, Zoe, Ion*

| | |
|---|---|
| `int compare(T a, T b)` | negativ / 0 / pozitiv |
| `Collections.sort(lista, c)` | ordinea lui c |
| `lista.sort(c)` | la fel, pe listă |
| `new TreeSet<>(c)` | set ordonat cu c |
| `new TreeMap<>(c)` | chei ordonate cu c |

```
class DupaNume implements
    Comparator<Elev> {
  public int compare(
      Elev a, Elev b) {
    return a.getNume().compareTo(
        b.getNume());
  }
}
```

⚠ Același `Elev` , mai multe ordini, fără să-l modifici.

## Comparable vs Comparator · două capcane

| | Comparable | Comparator |
|---|---|---|
| Unde se scrie | în clasa elementului | în altă clasă |
| Metoda | `compareTo(altul)` | `compare(a, b)` |
| Câte ordini | una (cea naturală) | oricâte |
| Ce modifici | clasa elementului | nimic; dai comparatorul colecției |
| Când | ai o ordine firească | altă ordine sau clasă pe care n-o poți schimba |

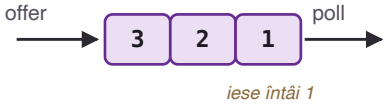
⚠ Element fără `Comparable` și fără comparator în `TreeSet` / `TreeMap` : **ClassCastException** la `add` / `put` .

⚠ În `TreeSet` / `TreeMap` , `compareTo == 0` (sau `compare == 0` ) = același element: al doilea nu intră, chiar dacă `equals` le spune diferite. `HashSet` merge pe `equals` / `hashCode` : intră amândouă.

```
Ana(9) și Bia(9) (aceeași notă; equals → false)
TreeSet (după notă): add → true, false → [Ana(9)]
HashSet: add → true, true → size() = 2
```

## Queue<E>

Primul intrat, primul ieșit (FIFO).




*iese întâi 1*

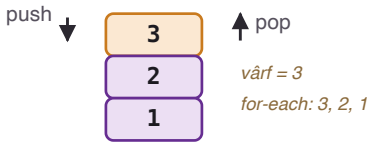
| | |
|---|---|
| `offer(x)` | la coadă → true |
| `poll()` | scoate din față (null dacă e goală) |
| `peek()` | se uită (null dacă e goală) |
| `size()` | câte așteaptă |

⚠ Familia care aruncă: `add`, `remove()`, `element()` → **NoSuchElementException** pe coadă goală.

```
Queue<String> q = new ArrayDeque<>();
```

## Deque<E> · ArrayDeque<E>

Coadă cu două capete: stivă (LIFO) și coadă, în aceeași clasă.




*vârf = 3*

*for-each: 3, 2, 1*

| | |
|---|---|
| `push(x)` | pune în vârf |
| `pop()` | scoate din vârf (aruncă dacă e goală) |
| `peek()` | se uită la vârf (null dacă e goală) |
| `offerFirst / offerLast` | la început / la sfârșit |
| `pollFirst / pollLast` | scoate de la capete (null dacă e goală) |

⚠ `Stack` și `Vector` sunt vechi: pentru stivă folosești `ArrayDeque`.

## Cele două familii de metode (Queue / Deque)

| Operația | Aruncă NoSuchElementException când e goală | Întoarce `null / false` |
|---|---|---|
| Adaugi | `add(x)` | `offer(x)` |
| Scoți | `remove()` | `poll()` |
| Te uiți | `element()` | `peek()` |
| Stivă | `pop()` | `peek()` |

## NU se folosesc (moștenire veche)

| Nu folosești | Scrii așa |
|---|---|
| `java.util.Stack` | `Deque<E> s = new ArrayDeque<>();` cu `push` / `pop` |
| `java.util.Vector` | `List<E> l = new ArrayList<>();` |